

生成AI活用の進化と仕様化

スライド説明書

杉田研治

Version 1.0.0

2026-08-04

この説明書について

この文書は、セミナー「生成AI活用の進化と仕様化」のスライドを、後から読み返すための 補助資料です。スライドだけでは省略されている背景、図表の読み方、各ページ間のつな ぎを説明します。

本文の「PDF ○ページ」は、配布スライド `seminar_slides.pdf` のページ番号を指します。スライドのPDF 1ページ目は表紙で、本文は2ページ目から始まります。

この説明書は、発表内容の理解を助けることを目的とします。アプリ生成ワークフローの正式 な利用手順ではありません。実際にワークフローを使用するときは、配布物の `USER_GUIDE.md` および `three_key_documents_workflow.md` を参照します。

全体像

セミナーは、生成AIの活用を次の流れで整理しています。

1. 1回の指示を工夫します。
2. 作業に必要な文脈を構造化します。
3. 文書、手順、レビューを再利用可能な枠組みにします。
4. 実行結果を検証し、作業方法の改善へ戻します。
5. 仕様化によって、人が判断する範囲とAIに任せる範囲を分けます。
6. 成果物を作るAIと、作業の境界を管理するAIを分けます。

スライド別説明

PDF 1ページ: 表紙

題名の「生成AI活用の進化」は、AIの性能向上そのものではなく、AIを利用する側の作業 設計が発展していくことを指します。副題は、その発展を Prompt engineering から Teacher-Worker engineering まで順に説明することを示しています。

PDF 2ページ: 頼み方から作業構造へ

このページは、セミナー全体の問題意識を示しています。単発の依頼では「うまく頼む」こと が中心になりますが、業務で繰り返し利用するには、文脈、成果物、確認方法、失敗時の戻り先 ま で含めて作業構造を設計する必要があります。

図の最後にある「改善ループへ戻す」は、今回見つかった問題をその場の修正だけで終わら せず、次のテンプレートや手順の改善へ反映することを表します。

PDF 3ページ: 50分で共有する到達点

4つの観点は独立した話題ではなく、後の項目が前の項目を支える関係にあります。AI活用の 段階を整理し、仕様化によって判断範囲を明確にし、それをワークフローとして再利用し、最後に作る役割と守る役割を分けます。

このセミナーで中心となる主張は、AIに任せる範囲を広げるほど、人が管理する正本、境界、完了条件を明確にする必要があるという点です。

PDF 4ページ: 単発利用から作業構造へ

左列は、依頼ごとにその場で調整する運用で起きやすい問題です。右列は、それらを個人の経験だけに頼らず、再現可能にするために事前に決める内容です。

特に「失敗がその場限りになる」と「戻し先を決める」の対応が重要です。出力だけを直すと、同じ前提で再生成した際に問題が再発します。問題の発生源となった要求、仕様、制約、手順へ戻すことで、次の作業にも修正を残せます。

PDF 5ページ: AI活用の4段階

Prompt、Context、Harness、Loop は成熟度の単純な順位ではありません。後ろの段階へ進んでも、明確なプロンプトは必要です。違いは、設計対象が1回の入力から、文脈、作業枠組み、反復と改善へ広がる点にあります。

この4段階は、現場で不足しているものを見つける見取り図として使えます。回答が曖昧ならPrompt、前提が欠けるならContext、毎回手順が異なるならHarness、同じ失敗が繰り返されるならLoopの観点で点検します。

PDF 6ページ: Prompt engineering

Prompt engineering は、1回の入力でAIの出力を安定させる工夫です。条件、形式、役割、例、禁止事項を明示すると、AIが推測する範囲を狭められます。

一方、既存仕様、過去の判断、変更禁止範囲などを毎回一つのプロンプトへ詰め込むと、記載漏れや更新漏れが起きます。Prompt engineering は重要な入口ですが、継続作業の状態を単独で管理する仕組みではありません。

PDF 7ページ: Context engineering

Context engineering では、要求、既存仕様、制約、レビュー観点など、作業に必要な情報を選り、構造化してAIへ渡します。

情報量を増やすだけでは十分ではありません。どの文書が正本か、どの判断が現在も有効か、何を変更してよいかを明示しなければ、古い情報や相互に矛盾する情報まで判断材料になります。文脈の品質は、量だけでなく、選別、構造、更新状態で決まります。

PDF 8ページ: Harness engineering

ここでいうハーネスは、AIが一定の条件で作業できるようにする外側の枠組みです。文書 群、テンプレート、工程、レビュー観点、完了条件を組み合わせます。

アプリ生成ワークフローは、このハーネスの具体例です。誰が実行しても同じ順序と基準 を利用でき、問題をどの工程へ戻すかを共有できるようにします。目的はプロンプトを不要に することではなく、プロンプトを再利用可能な作業構造の中へ位置付けることです。

PDF 9ページ: Loop engineering

上段の青い循環は、今回の成果物を完了させる内側の実行ループです。実行、検証、修正 を繰り返し、定めた完了条件を満たしたところで止めます。

下段の橙色の流れは、複数の作業を通じてハーネスを改善する外側のループです。検証で 見つかった問題のうち、繰り返し起きるものを分類し、手順、テンプレート、レビュー観点、完了条件へ反映します。

現在のアプリ生成ワークフローには前工程へ戻る土台があるが、開始条件、停止条件、状態 管理、自動反復までをすべて明示した仕組みではありません。まず手動で安定させ、定型化できる部分を後から自動化する段階にあります。

PDF 10ページ: 仕様不在によるAIコンテキスト劣化

仕様や設計意図がない場合、AIはコード、コメント、命名、会話履歴から目的を推定します。この推定は一見もっともらしくても、意図された仕様とは限りません。

表の4概念は、この問題を異なる角度から捉えるための手掛かりです。ここで重要なのは 用語の厳密な同一性ではなく、文脈の陳腐化、要求の曖昧さ、暗黙知の喪失、仕様を一次成 果物として扱う必要性が、AIを用いた開発でも相互に関係する点です。

PDF 11ページ: AIの補完リスク

生成AIは不足した情報を補って、作業を先へ進められます。この性質は便利ですが、人が決めるべき事項まで補完すると、仮の判断が仕様のように実装されます。

未決事項、対象外、実装制約を明記するのは、AIの能力を抑えるためではありません。判断してよい範囲を示し、補完が有効な場所と、人へ確認すべき場所を分けるためです。

PDF 12ページ: コードだけを正本にしない

表は、問題の種類と修正先の対応を示しています。たとえば画面操作の意図違いをコードだけで直すのではなく、UI仕様書へ戻してからコードまで整合させます。

戻り先を残すと、「何が間違っていたか」だけでなく「どの判断を直したか」を追跡できます。これにより、再生成、担当者交代、別のAIによる作業でも同じ判断を引き継ぎやすくなります。

PDF 13ページ: 人とAIの判断範囲

図は、人が目的や重要事項を判断し、それを仕様として明示し、AIが仕様に従って作業し、最後に人が結果をレビューする流れです。

人のレビューは、形式確認だけではありません。AIが置いた仮定、対象外への越境、業務判断、安全性、採用可否を確認します。仕様化は、人の判断をAIへ置き換える作業ではなく、両者の境界を見える形にする作業です。

PDF 14ページ: 要求からレビューまでを分ける

要求、仕様、制約、実装、テスト、レビューは互いに関係しますが、役割が異なります。混在させると、実装上の都合が仕様になったり、コードの現状がテストの期待値になったりします。

文書を分ける目的は文書量を増やすことではありません。判断の種類と戻り先を分けることです。小規模な作業では各文書を短くできますが、この役割の区別は維持します。

PDF 15ページ: アプリ生成ワークフローの全体

このワークフローは、主にUIを持つアプリケーションを対象とします。要求から直接実装せず、要求レビュー、仕様化前ノート、三点セット、実装プロンプトを経てコードへ進みます。実装後にもテスト、実装結果レビュー、設計文書化を置きます。

実装設計契約は仕様とコードの対応を追跡し、アプリ設計書はレビュー済みの実装を基に全体設計を記録します。重大な不整合が残る場合は、後工程の文書を完成扱いにせず仕様へ戻します。

PDF 16ページ: 既存工程への部分適用

ワークフロー全体を一度に導入する必要はありません。現在の問題に対応する部分だけを既存工程へ追加できます。

たとえば要求の曖昧さが問題なら要求定義レビューから始め、UIと業務ルールの混線が問題なら三点セットを使います。生成済みコードを既存の設計工程へ接続したい場合は、実装結果レビュー、実装設計契約、アプリ設計を利用します。

PDF 17ページ: 仕様化前ノート

仕様化前ノートは、詳細仕様を書く前に判断材料を整理する軽量の正本です。目的、利用者、入力、表示、ルールだけでなく、対象外と未決事項も明示します。

対象外は過剰実装を防ぎ、未決事項はAIによる仮決めを防ぎます。要求が大きい場合は単一文書へ詰め込まず、分割した仕様化前ノート群を正本として扱います。

スライドにある記述量の割合は、本ワークフローで用いた文書例を基にした比較で、すべての案件で同じ比率になるという意味ではありません。

PDF 18ページ: 三点セット

三点セットは、UIを持つアプリの仕様を、利用者の目的、画面と操作、画面から独立した業務処理へ分けます。

UC仕様書は利用者視点、UI仕様書は画面視点、ビジネスレイヤー仕様書は処理規則の視点を担います。同じ振る舞いを異なる観点から記述するため、三文書間の整合性をAIと人の順でレビューしてから実装へ進みます。

PDF 19ページ: 重要ロジックの括り出し

三点セットですべてを表現する必要はありません。金額計算、状態遷移、権限判定など、誤りの影響が大きく厳密な境界を必要とする処理は、ヘッダ定義やVDM-SLなどの別仕様へ括り出します。

仕様化前ノートには詳細を重複させず、括り出す対象、理由、元の業務仕様、別仕様の保存先、後続文書への反映先を追跡情報として残します。別仕様は人がレビューしてから実装します。

これは現在のワークフローへ完全に組み込み済みの手順ではなく、重要ロジックの別仕様化とTeacher-Workerを組み合わせる今後の運用案です。

PDF 20ページ: 実装プロンプト

実装プロンプトは、レビュー済み仕様をコード生成作業へ受け渡す文書です。参照する正本、作業ディレクトリ、実装対象、制約、対象外、ビルド条件などを明示します。

仕様書が「何を満たすか」を定義するのに対し、実装プロンプトは「どの資料と条件に従って実装するか」を定義します。未決事項が残る場合はAIに仮決めさせず、質問または未解決事項として報告させます。

PDF 21ページ: 意図違いの戻し方

生成結果が意図と違う場合、最終成果物にあたるコードだけを修正しません。まず、意図の起点となる仕様化前ノートを確認し、三点セット、実装プロンプト、コードの順で整合させます。

すべての問題で必ず最上流まで戻るという意味ではありません。実装方式だけの問題なら実装プロンプトへ戻ります。重要なのは、問題の種類を分類し、その判断を保持する最も上流の正本から直すことです。

PDF 22ページ: Teacherに統合する役割

Teacher-Worker engineering は、計画と実行、案内と実装、監督と作業、評価と作成を分ける既存の考え方と重なります。

新しい個別役割を発明したという主張ではありません。これらの機能を、Workerより上位の作業文脈を管理するTeacherという立場に統合し、状況に応じて使い分けることが中心です。

PDF 23ページ: 人、Teacher、Workerの受け渡し

作業は、人が目的、背景、期待する成果、制約、対象外をTeacherへ伝えるところから始まり、Teacherが目的を自律的に決める運用ではありません。

Teacherは `worker_task.md` に、今回の目的、参照する正本、変更可能範囲、対象外、完了条件、報告項目をまとめます。Workerは `worker_report.md` に、成果物、検証結果、仕様との差分、仮定、未解決事項、判断が必要な内容を返します。

Teacherの会話履歴全体をWorkerへ渡さず、作業に必要な文脈だけを文書化することで、役割ごとのコンテキストを分離します。重要事項の採用と最終判断は人に残します。

PDF 24ページ: Teacherの一般化

Teacherは一つの固定的な職能名ではなく、Workerより上位の作業文脈を管理する立場です。工程を決めるときはPlanner、戻り先を示すときはNavigator、境界を守るときはGuardian、結果を評価するときにはReviewer、判断理由を伝えるときはMentorとして働きます。

Mentorとして理由を伝えるのは、説明を増やすためではありません。判断理由を次の作業へ引き継ぎ、同じ失敗を繰り返さないようにするためです。

PDF 25ページ: 成果物作成と境界管理を分ける理由

成果物を作るAIが仕様やハーネスも変更できると、目の前の作業を完成させるために境界を動かす可能性があります。実装しやすいように仕様を変える、制約を緩める、対象外を追加するといった混線です。

Teacherは問題、戻り先、守るべき制約、判断理由を示し、成果物を直接修正しません。実際の修正はWorkerへ返します。この分離により、成果物の作成と評価、個別修正とハーネス改善を区別します。

PDF 26ページ: 生成対象と生成対象外

AIが繰り返し生成してよい領域と、上書きしてはいけない領域を分けます。アプリ本体や画面コードは生成対象にできる一方、共通ライブラリ、プラットフォーム適応層、重要ロジックの仕様は境界の外で管理します。

重要ロジックを別Workerが実装する場合、アプリ本体のWorkerには公開ヘッダーから利用させ、内部を変更させません。生成単位を分けることで、別の目的で蓄積した知見や厳密な仕様を、アプリ側の都合による再生成から守ります。

PDF 27ページ: Consultant AI

Consultant AIは、Teacher-Workerの恒常的な第三役割ではありません。現在の作業文脈へ直接混ぜたくない技術調査、事実確認、専門的助言を、一時的な別コンテキストで行う相談役です。

Consultantは成果物やワークフローを直接変更しません。回答を人へ返し、人が必要性を判断したうえで、必要な内容だけをTeacherまたはWorkerの文脈へ反映します。

表の研究例は、名称、専門家の動的追加、役割と会話パターンの構成という近い点を示したもので、ここで定義するConsultant AIと完全に同じ仕組みという意味ではありません。

PDF 28ページ: 全体のまとめ

Promptは入力、Contextは文脈、Harnessは作業枠組み、Loopは反復と改善を設計対象とします。仕様化は、人とAIの判断範囲を分けます。Teacher-Workerは、成果物を作る役割と、その作業の方向、境界、評価基準を守る役割を分けます。

これらは独立した流行語ではなく、AIを業務で繰り返し利用するときに必要な設計対象を、段階的に整理したものです。

PDF 29ページ: AI活用段階の参考整理

本編で扱った4段階を、既存概念との関係から再掲しています。Harness engineeringとLoop engineeringは比較的新しい実務寄りの整理で、組織や資料によって用語の範囲が異なる場合があります。

名称だけを導入するのではなく、自分たちの作業で何を正本とし、何を検証し、どこで止め、何を次回へ改善するかを具体化することが重要です。

PDF 30ページ: 仕様不在に近い研究概念

このページは、「仕様不在によるAIコンテキスト劣化」を調べる入口です。それぞれの研究概念が、セミナー内の用語と完全に同じ定義を用いているという意味ではありません。

Context Rotは文脈の古さ、Requirement Ambiguityは要求の曖昧さ、Tacit Knowledge Loss は暗黙知の喪失、Spec-Driven Developmentは仕様を一次成果物として扱う考え方に対応します。複数の観点を組み合わせることで、仕様不在の問題を技術面と運用面から捉えられます。

PDF 31ページ: 一般化したTeacherの参考整理

この表は、既存手法全体をTeacher-Workerへ置き換えるものではありません。既存の役割分離が持つ機能のうち、工程設計、作業案内、境界管理、評価、知識継承をTeacherの立場から整理しています。

実際の運用では、一人の人がTeacherの判断を担い、AIが形式確認や分類を補助する形でも構いません。Teacherは特定の製品やモデル名ではなく、上位文脈を管理する役割を指します。

PDF 32ページ: AI活用と仕様の参考リンク

Prompt、Context、Harness、Loopと、仕様不在に関係する研究を調べるためのリンク集です。本編では概念間の関係を理解することを優先し、個々の研究の詳細説明は省略しています。

リンク先を読むときは、発表時点以降に内容や版が更新されている可能性があることに注意します。また、論文や記事で使われる用語の定義と、本セミナーでの実務的な整理を区別して読みます。

PDF 33ページ: 知識、仕様、役割分離の参考リンク

暗黙知、仕様駆動、形式手法、複数の役割分離、Consultant AIに近い研究を調べるためのリンク集です。

VDM-SLは重要ロジックを厳密に記述する方法の一例として挙げています。すべてのロジックへ形式手法を適用するという意味ではなく、影響度や検証の必要性に応じて採用を判断します。

PDF 34ページ: Q/A

最後のページは、内容を自分の業務へ当てはめるための観点を示しています。適用を始めるときは、ワークフロー全体の導入から考える必要はありません。

まず、AIの出力がぶれる原因が、指示、文脈、手順、レビュー、改善のどこにあるかを確認します。そのうえで、要求定義レビュー、仕様化前ノート、三点セット、実装結果レビューなど、現在の問題に対応する部分から試します。

読後の確認ポイント

スライドと本書を読んだ後は、次の点を自分の作業に置き換えて確認してください。

- 現在の作業で正本となる文書は何でしょうか。
- AIが判断してよい範囲と、人が判断する範囲はどこでしょうか。
- AIが変更してよい対象と、変更してはいけない対象は何でしょうか。
- 成果物の完了を、どの方法と条件で確認しますか。
- 問題が見つかったとき、どの要求、仕様、制約、手順へ戻しますか。
- 繰り返す問題を、次回のテンプレートやレビュー観点へどう反映しますか。